

Objetivo

Implementar la integración entre la **capa de presentación** y las **APIs desarrolladas en la capa de aplicación**, garantizando un consumo eficiente, desacoplado y mantenible de los servicios backend, bajo principios de arquitectura limpia y diseño orientado a servicios.

Contexto

Esta práctica forma parte del proyecto final del curso y tiene como propósito consolidar la implementación de arquitecturas modernas basadas en APIs.

Los estudiantes deberán consumir las APIs previamente desarrolladas (RESTful en ASP.NET Core o similar) desde la interfaz de usuario (ASP.NET MVC, Razor, React u otra), aplicando buenas prácticas como:

- Separación de responsabilidades (Presentation vs API)
 - Uso de DTOs para comunicación
 - Manejo de errores y estados HTTP
 - Validaciones distribuidas (cliente y servidor)
 - Diseño estructurado de la capa de presentación
-

Actividades

1. Diseño de la integración con APIs

- Identificar los endpoints disponibles en la API:
 - GET (consultas)
 - POST (creación)
 - PUT/PATCH (actualización)
 - DELETE (eliminación)
- Definir los modelos de consumo (DTOs de entrada/salida)
- Diseñar la comunicación:
 - HTTP Client / HttpClientFactory (.NET)

- Fetch / Axios (Frontend JS)
 - Establecer contratos claros:
 - JSON como formato estándar
 - Manejo de códigos HTTP (200, 400, 404, 500)
-

2. Diseño de la arquitectura lógica de la capa de presentación 🧩

Los estudiantes deberán **definir y documentar la arquitectura interna de la capa de presentación**, incluyendo:

◆ Componentes mínimos esperados:

- **Controladores / Pages / Components**
 - Orquestan la interacción del usuario
- **Servicios de consumo de API**
 - Encapsulan llamadas HTTP (NO llamadas directas desde controladores)
- **ViewModels / Models**
 - Representan los datos que se muestran en la UI
- **Helpers / Utilities (opcional)**
 - Manejo de formatos, validaciones, etc.

◆ Diagrama requerido

Deben construir un **diagrama de arquitectura lógica** que represente:



◆ Buenas prácticas a evaluar

- Separación clara de responsabilidades
- Bajo acoplamiento
- Reutilización de servicios
- No mezclar lógica de negocio en la UI

3. Implementación técnica

Consumo de APIs

- Implementar servicios en la capa de presentación.

Flujo de interacción

1. Usuario interactúa con la UI
2. Controlador / Componente invoca servicio API
3. API procesa lógica de negocio
4. Retorna respuesta (JSON)
5. UI renderiza resultados

Integración con la UI

- ASP.NET MVC:
 - Controladores consumen servicios API
 - Uso de ViewModels
- React / Frontend:
 - Hooks (useEffect)
 - Manejo de estado (React Query / Redux)

Validaciones

- Frontend:
 - Required fields
 - Validaciones de formularios
- Backend:
 - Validaciones en API (FluentValidation / DataAnnotations)

Renderizado

- Uso de:
 - Partial Views (MVC)
 - Componentes reutilizables (React)

4. Pruebas funcionales

- Validar:
 - Consumo correcto de endpoints
 - Manejo de errores (API caída, timeout, etc.)
 - Flujo completo UI → API → UI
- Casos:
 - Datos válidos
 - Datos inválidos
 - API no disponible

Entregables

Código fuente

- UI integrada con consumo de APIs
- Servicios desacoplados correctamente implementados

Evidencia de funcionamiento

- Video o capturas mostrando:
 - CRUD vía API
 - Validaciones
 - Manejo de errores

Documento técnico (máx. 3 páginas)

Debe incluir:

1. **Arquitectura de la solución**
2. **Arquitectura lógica de la capa de presentación (OBLIGATORIO)**
 - Diagrama
 - Explicación de componentes
3. **Diseño de integración con APIs**
4. **Estrategia de consumo**
5. **Conclusiones**